

DhakaSim (Python) Codebase Analysis

Deep Technical Guide & Scenario Extension Manual

DhakaSim Research Team

July 25, 2026

Contents

1	Project Overview	4
2	The Strip Model	4
2.1	Strip allocation	4
2.2	Worked example — the shipped Dhaka network	5
2.3	Vehicle footprint	5
2.4	Longitudinal position	6
3	Architecture — How the Simulator Works	7
3.1	Execution flow	7
3.2	Startup sequence	8
3.3	Layering	8
4	Repository Layout	8
5	Module Reference	10
5.1	Network topology layer	10
5.2	Traffic entity layer	11
5.3	Simulation engine layer	11
5.4	Support layer	12
5.5	Presentation & tooling	12
6	The Simulation Loop, Step by Step	12
6.1	Inside <code>_move_vehicles()</code>	14
6.2	Ordinary driving — <code>Vehicle.move_vehicle_in_segment()</code>	14
6.3	Moving forward — <code>Vehicle._move_forward_in_segment()</code>	14
7	Vehicle Generation & Demand	15
7.1	From <code>demand.txt</code> to a spawn schedule	15
7.2	Arrival process	15
7.3	Placing the vehicle	15
7.4	Vehicle type mix	16
8	Vehicle Types	16

9	Car-Following Models	16
9.1	Gap definition	16
9.2	Gipps / hybrid (the default)	17
9.3	MY_MODEL — the strip-aware model	18
9.4	The remaining models	18
10	Lane-Changing (Strip-Changing) Models	19
11	Side Friction: Pedestrians & Roadside Objects	19
11.1	Road-crossing pedestrians (<code>AcrossPedestrianMode</code>)	20
11.2	Along-road pedestrians (<code>AlongPedestrianMode</code>)	20
11.3	Roadside objects (<code>ObjectMode</code>)	20
12	Intersections & Signals	21
12.1	Turn paths are created lazily	21
12.2	Signals	21
12.3	Entering, traversing and leaving	21
13	The Input File System — Scenario Specification	22
13.1	<code>node.txt</code> — intersections and boundaries	22
13.2	<code>link.txt</code> — road geometry	22
13.3	<code>path.txt</code> — routes (generated)	22
13.4	<code>demand.txt</code> — traffic volume matrix	23
13.5	<code>parameter.txt</code> — runtime configuration	23
13.6	Regenerating routes and demand	23
14	Complete Parameter Reference	23
14.1	Run control	23
14.2	Geometry & display	24
14.3	Behavioural models	24
14.4	Demand	24
14.5	Side friction	25
14.6	Signals & measurement	25
15	Output Files & Metric Definitions	25
15.1	Standard output	25
15.2	<code>statistics/*.csv</code>	26
15.3	The fuel model	26
15.4	<code>trace.txt</code>	27
16	The Numeric Compatibility Layer	27
17	Known Behaviours & Quirks	28
18	How to Simulate a New Location	29
19	Simulating Road Diversions & Strip Blocking	31
19.1	Full closure	31
19.2	Road narrowing (construction, flyover works)	31
19.3	Static kerbside encroachment	31
19.4	Scripted obstacle (accident, dumped material)	31
19.5	Closing individual strips	31

20 Extension Recipes	32
20.1 Change the vehicle mix	32
20.2 Add a vehicle type	32
20.3 Add a car-following model	32
20.4 Use adaptive signals	32
20.5 Per-vehicle trajectory output	32
20.6 Make runs reproducible	32
21 Performance	33
22 Provenance & Validation	33
22.1 Java class → Python module map	33
23 Quick Reference	34
23.1 Commands	34
23.2 Key constants	34
Further Reading	34

1 Project Overview

DhakaSim is a specialised, event-based microscopic traffic simulator designed to capture the non-lane-based and heterogeneous traffic common in developing cities such as Dhaka, Bangladesh. Unlike traditional lane-based microscopic simulators (e.g. SUMO, VISSIM) that assume vehicles follow single discrete lanes, DhakaSim divides each road into many fine-grained “strips” (≈ 0.5 m wide).

Vehicles occupy several adjacent strips according to their physical width. That single design decision is what makes the following behaviours emerge naturally rather than needing special cases:

- **filtering** — a motorbike (2 strips) slipping between a bus (5 strips) and the kerb;
- **continuous lateral placement** — vehicles drift sideways strip by strip instead of snapping to lane centres;
- **side-friction bottlenecks** — parked cars, rickshaws and CNGs occupying the outer strips, permanently narrowing the usable carriageway;
- **mixed-speed interaction** — motorised traffic (car, bus, truck, CNG, motorbike) sharing strips with non-motorised traffic (rickshaw, pushcart, bicycle) and with pedestrians walking along and across the road.

This document describes the **Python implementation**, a faithful translation of the original Java simulator; see Section 22.

Property	Value
Execution model	fixed time-step, 1 simulated second per step (<code>Constants.TIME_STEP</code>)
Spatial resolution	0.5 m strips (<code>StripWidth</code>)
Vehicle types	13 (including along-road pedestrians as “type 12”)
Car-following models	13 selectable
Lane-changing models	4 selectable
Dependencies	none — standard library only, <code>tkinter</code> for the GUI
Python	3.10+
Code size	$\approx 8,200$ lines across 29 files

Table 1: Key characteristics of the simulator.

2 The Strip Model

Everything in DhakaSim is built on the strip. Understanding the indexing scheme makes the rest of the codebase readable.

2.1 Strip allocation

A **Segment** of physical width w metres is divided into

$$\text{stripCount} = 2 + \left\lfloor \frac{w - 2 w_{\text{footpath}}}{w_{\text{strip}}} \right\rfloor$$

Index 0 and index $\text{stripCount} - 1$ are **footpath strips** (pedestrians and roadside objects live here first); everything between is carriageway, split into two directions of travel:

$$\begin{aligned} \text{middleHighStripIndex} &= \lceil \text{stripCount} / 2 \rceil \\ \text{middleLowStripIndex} &= \text{middleHigh} - 1 \quad \text{if stripCount is even} \\ &= \text{middleHigh} - 2 \quad \text{if stripCount is odd} \\ \text{lastVehicleStripIndex} &= \text{stripCount} - 2 \end{aligned}$$



Region	Strip range	Meaning
Footpath (near)	0	pedestrians, parked objects
Direction A	1 ... middleLowStripIndex	forward travel
Median	(odd stripCount only)	unused divider strip
Direction B	middleHighStripIndex ... lastVehicleStripIndex	reverse travel
Footpath (far)	stripCount-1	pedestrians, parked objects

Table 2: Strip regions within a segment.

2.2 Worked example — the shipped Dhaka network

With `StripWidth 0.5` and `FootpathStripWidth 0.5`:

Road width	Strips	Direction A	Direction B	Footpaths
24.5 m	49	1–23	25–47	0, 48
27.0 m	54	1–26	27–52	0, 53
32.0 m	64	1–31	32–62	0, 63

Table 3: Strip layout of the three road widths present in the shipped network.

A 27 m road therefore offers 26 strips per direction — enough for five buses abreast, or thirteen motorbikes.

2.3 Vehicle footprint

$$\text{numberOfStrips} = \left\lceil \frac{\text{vehicleWidth}}{\text{StripWidth}} \right\rceil$$

A vehicle at $\text{stripIndex} = k$ occupies strips $k \dots k + \text{numberOfStrips} - 1$ and registers itself in the vehicle list of every one of them (`Vehicle._occupy_strips`). All neighbour searches — leader, follower, gap checks, collision checks — iterate those strips. Leaving a strip is `Vehicle.free_strips()`; a lateral move is `Vehicle._change_strip()`.

2.4 Longitudinal position

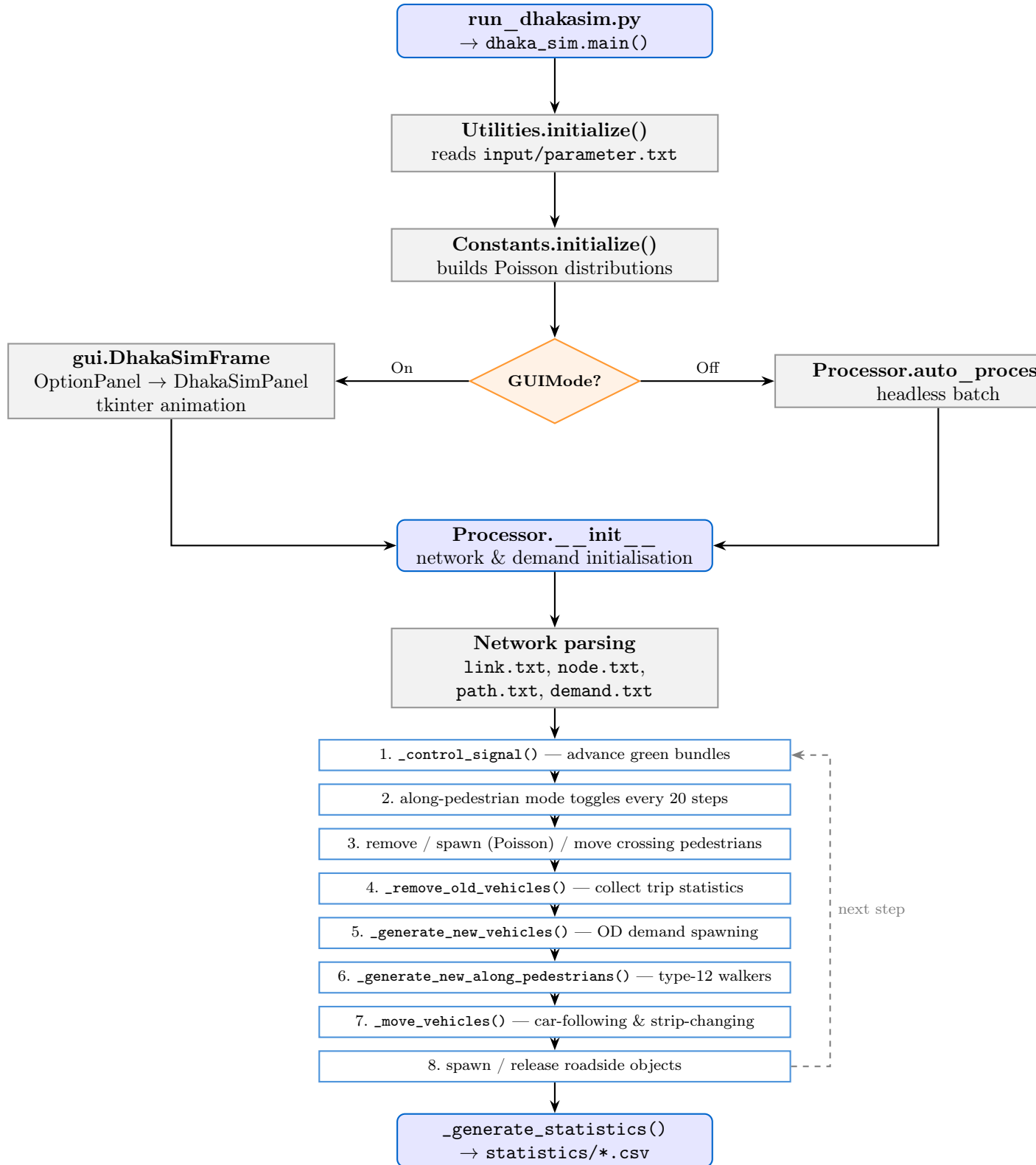
`distanceInSegment` is the position of the **tail** of the vehicle measured from the segment start, so the head is at `distanceInSegment + length`. A vehicle starts at 0.1 m and is considered at the segment end when

$$\text{distanceInSegment} + \text{length} + \text{MARGIN} \geq \text{segmentLength}, \quad \text{MARGIN} = 1 \text{ m}$$

For a vehicle travelling in direction B (`reverseSegment = True`) the geometry is mirrored when drawing and when comparing against pedestrian and object positions, but `distanceInSegment` still counts up from that vehicle's own entry end.

3 Architecture — How the Simulator Works

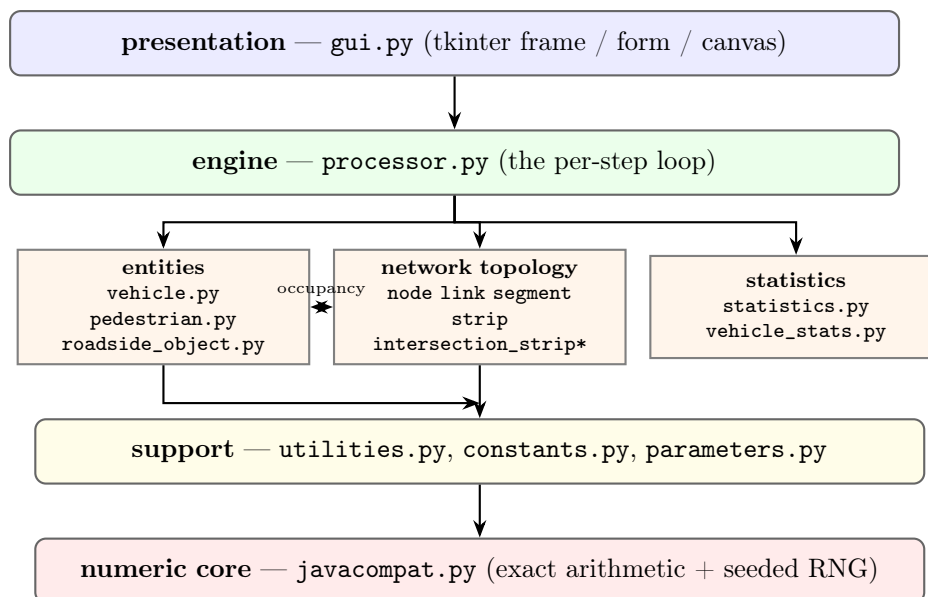
3.1 Execution flow



3.2 Startup sequence

1. `run_dhakasim.py` puts the package on `sys.path` and calls `dhakasim.dhaka_sim.main()`.
2. `Utilities.initialize()` parses `input/parameter.txt` into the `Parameters` class attributes: strip geometry, pixel scaling, active behavioural models, and the random seed.
3. `Constants.initialize()` builds the two Poisson pedestrian-arrival distributions. This is deliberately separate from module import, because the distribution means come from `parameter.txt` and so cannot exist until step 2 has run.
4. `Utilities.index_trace_file()` indexes `trace.txt` when `TraceMode On`, so the GUI's trace slider can seek to any step.
5. **Mode branching** on `Parameters.GUI_MODE` (overridable with `-headless / -gui`):
 - **GUI:** `DhakaSimFrame` shows `OptionPanel`; pressing *Start Simulation* applies the form values, creates `DhakaSimPanel`, and starts a `tkinter.after` timer that drives one step per tick.
 - **Headless:** `Processor().auto_process()` runs the loop to completion.
6. `Processor.__init__` reads the network, routes and demand, resets `Statistics`, creates `statistics/`, and computes the per-route generation schedule.

3.3 Layering



`javacompat.py` sits underneath everything: it supplies the arithmetic primitives (`jmin`, `jround`, `jdiv`, `JavaRandom`, ...) that the whole simulator is built on. See Section 16.

4 Repository Layout

```

1 DhakaSIM_python/
2 |-- input/
3 |   |-- parameter.txt
4 |   |-- link.txt
5 |   |-- node.txt
6 |   |-- path.txt
   scenario definition (read at startup)
   all runtime settings
   links and their segments (geometry, width)
   intersections and boundary nodes
   routes (link sequences) per OD pair
  
```

```

7 | |-- demand.txt          vehicles/hour per OD pair
8 | |-- demand_all.txt     unthinned demand from run_sim.py
9 | |-- demand-low/medium/high.txt alternative demand levels
10 |-- dhakasim/           the simulator package
11 |-- run_dhakasim.py     launcher
12 |-- run_sim.py         route + demand generator (Floyd-Warshall)
13 |-- tests/test_javacompat.py numeric regression tests
14 |-- README.md          user-facing quick start
15 |-- DhakaSIM_Python_Codebase_Analysis.{md,tex} this document

```

Listing 1: Repository layout

Generated at run time (git-ignored): `statistics/`, `trace.txt`, `debug/`.

5 Module Reference

5.1 Network topology layer

Module	Lines	Purpose / Functionality
node.py	277	Intersections. Owns <code>IntersectionStripBundles</code> (one per incoming link) carrying signal state, the list of <code>IntersectionStrips</code> (turn paths through the junction), and the vehicles currently inside. Runs separating-axis collision detection between vehicles in the junction. A node with more than one link is a signalised junction.
link.py	57	Connects two nodes (<code>upNode</code> , <code>downNode</code>); an ordered list of <code>Segments</code> .
segment.py	293	A straight road stretch: start/end coordinates, width, the strip array, a mid-point sensor, and per-segment counters (throughput, average speed, waiting time, accidents). Computes the strip layout of Section 2.
strip.py	528	The spatial unit. Holds the vehicle, pedestrian and object lists occupying it, and answers every neighbour query: <code>probable_leader</code> , <code>probable_follower</code> , <code>probable_object_leader</code> , <code>has_gap_for_adding_vehicle</code> , <code>has_gap_for_strip_change</code> (where the lane-changing models live), <code>get_gap_for_forward_movement</code> , <code>has_collision_occurred</code> , <code>check_for_accident</code> .
intersection_strip.py	35	One turn path across a junction: entry strip $\rightarrow$ exit strip, with the two pixel-space endpoints defining its centre line.
intersection_strip_bundle.py	58	All turn paths that <i>enter</i> the junction from one link, sharing a single signal state. Signal control operates on bundles, not individual turns.
link_segment_orientation.py	45	Given a point and a link, decides whether the vehicle traverses the link forwards or backwards (<code>reverseLink</code>) and whether the segment is entered from its start or its end (<code>reverseSegment</code>). Everything directional keys off these two flags.

5.2 Traffic entity layer

Module	Lines	Purpose / Functionality
<code>vehicle.py</code>	2,309	The main entity. Physical properties from type; longitudinal control (13 car-following models); lateral control (strip changing); junction traversal including deadlock escape; fuel consumption; collision/accident bookkeeping; drawing geometry.
<code>pedestrian.py</code>	218	A road-crossing pedestrian: moves laterally across the strips one at a time (<code>move_forward</code>), and shuffles longitudinally (<code>move_length_wise</code>) when blocked. Removed on reaching the far side or when stuck longer than <code>segmentWidth/speed</code> .
<code>roadside_object.py</code>	207	A stationary side-friction element: standing pedestrian, parked car, parked rickshaw, parked CNG. Occupies a contiguous block of strips from the footpath inwards for a randomly drawn parking duration.

Pedestrians walking **along** the road are not `Pedestrian` objects — they are `Vehicles` of type 12 with a 0.4×0.4 m footprint. This lets them participate in car-following and strip-changing like any other road user.

5.3 Simulation engine layer

Module	Lines	Purpose / Functionality
<code>processor.py</code>	1,367	The engine. Network/route/demand parsing, per-step orchestration, vehicle and pedestrian and object generation, signal control, junction entry/exit, flow measurement, final statistics output.
<code>parameters.py</code>	98	Every runtime setting as class attributes (Java <code>static</code> fields), plus the three model enumerations.
<code>constants.py</code>	130	Fixed physical constants, colours, and the empirical Gaussian-mixture blockage distributions for the four roadside object types, derived from Dhaka field survey data.
<code>statistics.py</code>	60	Global accumulators, reset per run.
<code>vehicle_stats.py</code>	37	Per-vehicle speed and trajectory time series (written out only when <code>Processor.write_speed()</code> is enabled).
<code>dhaka_sim.py</code>	37	Entry point: initialise, then branch on GUI mode.

5.4 Support layer

Module	Lines	Purpose / Functionality
<code>utilities.py</code>	658	Parameter file parsing; 2-D geometry helpers (<code>return_x3 ... return_y6</code> compute the perpendicular offsets used for road and vehicle corners); the error function and Gaussian CDF; per-type vehicle dimensions and performance; blockage sampling from the Gaussian mixtures; the strip-weight function used by the strip-based car-following model.
<code>javacompat.py</code>	640	Exact numeric semantics and the seeded RNG. See Section 16.
<code>demand.py</code>	40	An OD pair with its flow rate and the routes serving it.
<code>path.py</code>	33	A route: source node, destination node, ordered link ids.
<code>point2d.py</code> , <code>signal.py</code> , <code>normal_distribution.py</code>	14/16/39	A mutable 2-D point; the RED/GREEN/YELLOW enum; a single Gaussian component of a mixture.

5.5 Presentation & tooling

Module	Lines	Purpose / Functionality
<code>gui.py</code>	549	<code>DhakaSimFrame</code> (window), <code>OptionPanel</code> (parameter form), <code>DhakaSimPanel</code> (animated canvas), and <code>CanvasGraphics</code> — an adapter applying the pan/zoom transform and drawing onto a <code>tkinter.Canvas</code> . Also writes and replays <code>trace.txt</code> .
<code>run_sim.py</code>	281	Floyd–Warshall route generator: builds node adjacency from <code>link.txt/node.txt</code> , computes shortest paths between all boundary node pairs, writes <code>path.txt + demand.txt</code> .
<code>tests/test_javacompat.py</code>	131	Pins the numeric layer to reference values.

6 The Simulation Loop, Step by Step

`Processor._run_at_each_time_step()` executes exactly this order once per simulated second. The order matters — several stages depend on state written by an earlier one.

#	Call	What happens
1	<code>_control_signal()</code>	Only when step mod SignalChange 0. Advances the green bundle at every signalised node.
2	<i>(toggle)</i>	If along-pedestrian mode was on at startup, <code>along_pedes</code> flips every 20 steps, producing alternating 20-second bursts of foot-path pedestrian traffic.
3	<code>_remove_old_pedestrians()</code> <code>_generate_new_pedestrians()</code> <code>_move_pedestrians()</code>	Only when <code>AcrossPedes</code> On. Drop

6.1 Inside `_move_vehicles()`

For each vehicle, in list order (which is creation order):

```
1 if PenaltyWait and vehicle already collided:
2     if penalty time elapsed: remove from simulation
3     skip this vehicle
4 if vehicle is inside a junction:
5     free its strips (it is not on a segment)
6     if at junction exit: move_vehicle_at_intersection_end() # enter next
7     link
8     else:
9         move_vehicle_in_intersection()
10        then re-check for arrival at the exit
11 else:
12     look up the signal ahead and store it on the vehicle
13     if at segment end: move_vehicle_at_segment_end() # next segment,
14     or junction
15     else:
16         move_vehicle_at_segment_middle() # ordinary
17     driving
18     then re-check for arrival at the end
19 increment fuel consumption; record speed and position
```

Listing 2: Movement dispatch (pseudocode)

Then two separate passes over all vehicles:

1. `accident_log_all_vehicle_at_last()` — appends a row to `statistics/accident_log.csv` for any vehicle that has collided or struck a pedestrian.
2. `accident_check_all_vehicles_at_last()` — intended to apply the collision penalty. See quirk 5: in practice it never fires, because the check in pass 1 consumes the pedestrian.

Finally, every 60 steps the minute’s `flowCount` is written into `Statistics.flow` and reset.

6.2 Ordinary driving — `Vehicle.move_vehicle_in_segment()`

For everything except type-12 pedestrians:

```
1 if is_object_in_proximity() or not move_forward_in_segment() or
2    is_slower_vehicle_in_proximity():
3     try_lane_change(priority_towards_middle=True)
```

Read it as: *if a parked object is ahead, or I could not move forward at all, or my leader is slower than I want to go — try to change strips*. The three conditions are evaluated left to right and `move_forward_in_segment()` has the side effect of actually moving, so the ordering is load-bearing. A lane change is attempted towards the road centre first, then towards the kerb — except that a vehicle will not move towards the kerb when a parked object is there.

Type-12 pedestrians take a different branch: with probability `PedestrianRandomLaneChangePercentage/100` they attempt a random lateral move (biased left with probability `PedestrianLeftBiasPercentage/101`), otherwise they walk forward and only swerve if blocked.

6.3 Moving forward — `Vehicle._move_forward_in_segment()`

1. `_control_speed_in_segment()` picks the new speed from the active car-following model, clamps it to `currentMaxSpeed`, and — if `BrakeHard On` — further clamps it to the smallest actually-available gap across all occupied strips.

2. If the speed is usable, the position advances by **Butcher's fifth-order quadrature** rather than $x + v\Delta t$:

$$\begin{aligned}\Delta v &= v_{\text{new}} - v_{\text{old}} \\ k_1 &= v_{\text{old}}, \quad k_3 = v_{\text{old}} + 0.25\Delta v, \quad k_4 = v_{\text{old}} + 0.50\Delta v \\ k_5 &= v_{\text{old}} + 0.75\Delta v, \quad k_6 = v_{\text{new}} \\ x_{\text{new}} &= x + \frac{1}{90} (7k_1 + 32k_3 + 12k_4 + 32k_5 + 7k_6) \Delta t\end{aligned}$$

3. The position is capped so the vehicle never overruns the segment end, and the mid-segment sensor flag is set the first time it is passed.
4. If the speed was zero, **waitingTime** and **waitingTimeInSegment** both increment — this is what the waiting-time statistics measure.

7 Vehicle Generation & Demand

7.1 From demand.txt to a spawn schedule

`Processor._read_demand()` reads `source dest vehiclesPerHour` and stores `vehiclesPerHour + 30` — a hard-coded offset applied to every route, so a file value of 100 becomes an effective 130 veh/h.

For each route:

$$\begin{aligned}\text{demandRatio} &= 3600/\text{demand} \\ \text{vehiclesToGenerate} &= 1 \quad \text{if demandRatio} > 1, \quad \text{else round}(1/\text{demandRatio}) \\ \text{nextGenerationTime} &= 1\end{aligned}$$

`vehiclesToGenerate` only exceeds 1 for demands above 3600 veh/h, so with normal inputs one spawn attempt is made per arrival event.

7.2 Arrival process

`_generate_new_vehicles()` fires when `nextGenerationTime` equals the current step, then repeats until the next arrival falls in a later step:

VehicleGenerationRate	Headway
0 — CONSTANT	$X = 3600/\text{demand}$ (deterministic)
1 — POISSON	$X = (3600/\text{demand}) \cdot (-\ln U)$, $U \sim \text{Uniform}(0, 1)$ — exponential inter-arrival times, i.e. a Poisson process

7.3 Placing the vehicle

`_create_a_vehicle()`:

1. Pick a route uniformly among those serving the OD pair.
2. Pick a vehicle type from the mix (below).
3. Resolve the entry segment and its orientation, then the legal strip window for the travel direction (`1 ...middleLow` or `middleHigh ...lastVehicle`).

4. Pick a random start strip in that window, then scan **upwards** for a strip run wide enough and clear enough for the vehicle (`has_gap_for_adding_vehicle`, which requires the first `THRESHOLD_DISTANCE + 0.08 + length` metres to be free of vehicles, objects and pedestrians). If nothing is found, scan **downwards**.
5. If no gap exists anywhere, the vehicle is simply **not created** — demand is lost rather than queued. This is how the model saturates.

7.4 Vehicle type mix

`_distributed_vehicle_type()` draws `ratio = randomInt(0..100)` and, after the percentage split is forced to slow 25 / medium 25 / fast 50, resolves to:

Band	Prob.	Sub-draw	Type
Slow (ratio < 25)	$\approx 24.8\%$	< 9	0 bicycle
		9..97	1 rickshaw
		≥ 98	2 van / pushcart
Medium ($25 \leq \text{ratio} < 50$)	$\approx 24.8\%$	< 83	7 CNG
		83..97	8–9 bus
		≥ 98	10–11 truck
Fast (ratio ≥ 50)	$\approx 50.5\%$	< 88	3 motorbike
		≥ 88	4–6 car

Table 4: Vehicle type composition actually generated.

The result is a heavily rickshaw- and motorbike-dominated stream, which is the point: it reproduces observed Dhaka composition rather than a Western vehicle mix. Three alternative mixes are present but unused — `_distributed_vehicle_type_miami()`, `_distributed_vehicle_type_bd_new`, `_random_vehicle_type()` — and can be swapped in at `_pedestrian_vehicle_distribution_type()`.

8 Vehicle Types

Defined in `utilities.py` (`_CAR_WIDTHS`, `_CAR_LENGTHS`, `_CAR_SPEEDS`, `_CAR_ACCELERATIONS`). Maximum braking is -6 m/s^2 for every type.

Two details:

- The effective ceiling is $\min(\text{MaximumSpeed}, \text{typeMaxSpeed})$, so the network speed limit can bind before the vehicle’s own capability does.
- Type 12 gets **per-individual variation**: its max speed and acceleration are each perturbed by a Gaussian with $\sigma = \text{mean}/5$, so a crowd of walkers has a realistic speed spread.

9 Car-Following Models

Selected by `CF_model` in `input/parameter.txt`. All are implemented in `vehicle.py`; each returns a new speed, from which acceleration is back-computed as $(v_{\text{new}} - v_{\text{old}})/\Delta t$.

9.1 Gap definition

Every model works from the same gap, in `Vehicle.get_gap()`:

$$\Delta x = x_{\text{leader}} - \text{THRESHOLD_DISTANCE} - (x_{\text{follower}} + \text{length}_{\text{follower}})$$

Type	Vehicle	Width (m)	Length (m)	Strips @0.5 m	Max speed		Accel
					(km/h)	(m/s)	(m/s ²)
0	bicycle	0.55	1.90	2	15	4.1666	0.42
1	rickshaw	1.20	2.40	3	8	2.2222	0.25
2	van / pushcart	1.22	2.50	3	7	1.9444	0.20
3	motorbike	0.60	1.80	2	100	27.7777	2.80
4	car	1.70	5.00	4	100	27.7777	3.00
5	car	1.76	4.55	4	60	16.6666	2.80
6	car	1.78	4.30	4	110	30.5555	3.34
7	CNG (auto-rickshaw)	1.40	2.65	3	40	11.1111	1.11
8	bus	2.40	10.30	5	30	8.3333	0.84
9	bus	2.30	9.50	5	40	11.1111	1.12
10	truck	2.44	7.20	5	35	9.7222	0.96
11	truck	2.46	7.50	5	45	12.5000	1.24
12	pedestrian (along road)	0.40	0.40	1	5.1	1.4166	0.10

Table 5: The 13 vehicle types. Column order in every per-type CSV.

CF_model	Model	Method
0	HYBRID (default)	<code>_get_new_speed_gipps_model</code>
1	NAIVE	inline — accelerate maximally, ignore everything
2	GIPPS	<code>_get_new_speed_gipps_model</code>
3	KRAUSS	<code>_get_new_speed_krauss_model</code>
4	GFM (generalised force)	<code>_get_new_speed_gfm_model</code>
5	IDM (intelligent driver)	<code>_get_new_speed_idm_model</code>
6	RVF (reduced visual field)	<code>_get_new_speed_rvf_model</code>
7	VFIAC	<code>_get_new_speed_vfiac_model</code>
8	OVCN (optimal velocity, memory)	<code>_get_new_speed_ovcn_model</code>
9	KFTM	<code>_get_new_speed_kftm_model</code>
10	HDM (human driver, multi-leader)	<code>_get_new_speed_hdm_model</code>
11	SBM (stochastic behavioural)	<code>_get_new_speed_sbm_model</code>
12	MY_MODEL (strip-based)	<code>_get_new_speed_my_model</code>

Table 6: Available car-following models.

i.e. bumper-to-bumper clearance minus a 0.5m standstill buffer. The gap can go **negative**, which is how overlaps (collisions) become detectable.

When `ErrorMode On`, `get_dx()` instead returns a *perceived* gap corrupted by sensor error — radar error for the immediate leader, GPS position error for further leaders — controlled by `FTMethod`, `NoOfReadings` and `MFactor`. This supports connected/automated-vehicle experiments.

9.2 Gipps / hybrid (the default)

Free-flow branch — how fast the driver *wants* to go:

$$v_a = v_n + 2.5 a_n T \left(1 - \frac{v_n}{V_n} \right) \sqrt{0.025 + \frac{v_n}{V_n}}$$

Car-following branch — the fastest speed from which the driver can still stop safely behind the leader:

$$v_b = dT + \sqrt{d^2T^2 - d \left(2\Delta x - v_nT - \frac{v_{n-1}^2}{\hat{d}} \right)}$$

where d is the follower’s maximum braking (-6 m/s^2), T the reaction time ($= 1 \text{ s} = \text{one time step}$), and $\hat{d} = (d_{\text{leader}} + d_{\text{follower}})/2$ the follower’s estimate of the leader’s deceleration. The new speed is `precision2(max(0, min( $v_a$ ,  $v_b$ )))`.

Important: the square root argument goes negative whenever the leader is too close to stop behind, making v_b **NaN**. Java’s `Math.min` and `Math.max` propagate NaN, so `max(0, min( $v_a$ , NaN))` is NaN, which `precision2` then truncates to **0** — an emergency stop. This is a load-bearing behaviour, not an accident; see Section 16.

The difference between GIPPS and HYBRID is only meaningful together with `BrakeHard`, which adds a hard gap-based clamp on top of the model speed.

9.3 MY_MODEL — the strip-aware model

This is the model designed for non-lane traffic, and the only one that uses the strip structure rather than treating the road as a single lane.

1. **Free speed:** $v_a = \min(\text{currentMaxSpeed}, v + a\Delta t)$.
2. **Local density:** count distinct vehicles and pedestrians within the vehicle’s own strips widened by $\pm \text{numberOfStrips}$, over the longitudinal window $[x, x + 2\text{length}]$, divided by 9.
3. **Adaptive attention:** if $\text{density}/10 < \text{DensityPercentage}/100$ the driver ignores adjacent strips (`sideStrips = 0`); otherwise it considers `SideStripsToConsider = 1` strip on each side. In free flow you watch only your own path; in congestion you watch your neighbours.
4. **Congested speed** over all leaders in the attention window:
 - `ConsiderMinimum On` → the **minimum** Gipps braking speed over all leaders (conservative);
 - `ConsiderMinimum Off` → a **weighted average**, where each leader’s weight is its lateral overlap with the follower $\times$ a normalised centrality term $\times$ a type weight (`PedestrianWeight` for pedestrians, 1 otherwise), capped by the closest leader’s speed.

The weight function is `Utilities.get_weight()`: a leader that only clips the edge of your path restrains you less than one squarely in front.

9.4 The remaining models

Each is a faithful implementation of its published form, retained for the comparative studies the simulator was built for:

- **KRAUSS** — safe-velocity formulation with a $\tau = 0.3 \text{ s}$ reaction time.
- **GFM** — generalised force model; separate attraction and repulsion ranges ($R_a = 15 \text{ m}$, $R_d = 80 \text{ m}$).
- **IDM** — desired dynamic gap $s^* = s_0 + \max\left(0, vT + \frac{v\Delta v}{2\sqrt{ab}}\right)$ with $a = 3$, $b = 4$, $T = 1.5 \text{ s}$, acceleration exponent 4.

- **RVF / VFIAC** — optimal-velocity variants that average the leader’s speed over a 4-step memory window (`TIME_WINDOW`).
- **OVC**M — optimal-velocity with a 3-step gap memory (`GAP_WINDOW`).
- **KFTM** — combines an IDM-like and a Gipps-like speed and takes the minimum, evaluated over up to three leaders within 300 m; tuned by `ALPHA`, `BETA`, `ETA`.
- **HDM** — human driver model with explicit multi-leader interaction over four leaders.
- **SBM** — stochastic behavioural model with Gaussian noise on the desired spacing.

When no leader exists, the vehicle follows a **dummy leader**: a stopped zero-length vehicle at the link end if the vehicle is in the last segment (speed 0 on red, 5 m/s on green), or a vehicle at 10⁸ m travelling at the speed limit otherwise. This is how signals brake traffic without any special-casing in the car-following code.

10 Lane-Changing (Strip-Changing) Models

Selected by `DLC_model`. The decision is made in `Strip.has_gap_for_strip_change()`, called for the target strip.

A move is only considered if the target strip is geometrically free: no vehicle whose extent ($\pm$ threshold) overlaps the mover’s, and no pedestrian inside its footprint. Beyond that:

<code>DLC_model</code>	Model	Rule
0	NAIVE (default)	Geometric clearance is sufficient — change immediately.
1	GHR	Feasible if the General-Motors/GHR acceleration with respect to the new leader (and of the new follower with respect to you) stays above maximum braking.
2	GIPPS	Feasible if the required deceleration for you <i>and</i> for the prospective follower stays above maximum braking.
3	MOBIL	Accept if the new follower’s acceleration stays above $b_{\text{safe}} = -4 \text{ m/s}^2$ and the politeness-weighted total acceleration gain exceeds $a_{\text{th}} = 0.1 \text{ m/s}^2$.

For GHR and GIPPS an additional **gap-acceptance probability** is applied:

$$P = 1 - e^{-\lambda(t_{\text{gap}} - t_{\text{safe}})}, \quad \lambda = 0.78$$

evaluated for both the lead and lag gaps and multiplied together; the change happens only if a uniform draw falls below P . This makes lane changing stochastic rather than deterministic.

The manoeuvre itself (`_move_to_higher_index_lane` / `_move_to_lower_index_lane`) walks one strip at a time towards the limit for the travel direction, and rolls back to the original strip if it cannot find a position from which it can also move forward.

Type-12 pedestrians always use NAIVE regardless of `DLC_model` (`Utilities.get_dlc_model`).

11 Side Friction: Pedestrians & Roadside Objects

This is what distinguishes DhakaSim from lane-based simulators, and the distributions below come from field surveys in Dhaka.

11.1 Road-crossing pedestrians (AcrossPedestrianMode)

- **Arrival:** per link, per step, a Poisson draw with mean `AcrossPedestrianPerHour/3600` (default $400/3600 \approx 0.111$ per link-second).
- **Longitudinal position:** a three-zone mixture over the crossable range [9 m, length – 9 m]. The range is split into its **first 20 %**, **middle 60 %** and **last 20 %**, drawn with probabilities **0.4 / 0.2 / 0.4** — so 80 % of crossings happen in the 40 % of the block nearest the two junctions, which is what is observed on the ground.
- **Speed:** 0.05 or 0.15 m/s (chosen uniformly) — deliberately slow, because this value is the lateral progress per second across the strips.
- **Side:** the near or far footpath, uniformly.
- **Movement:** `move_forward()` advances laterally into the next strip if that strip has a gap; if blocked, `move_length_wise()` shuffles along the road instead. A pedestrian stuck longer than `segmentWidth/speed` is removed.
- **Effect on traffic:** a pedestrian inside a vehicle’s braking envelope forces `get_gap_for_forward_movement` to return 0 — a full stop — and a pedestrian inside a vehicle’s footprint registers an accident.

11.2 Along-road pedestrians (AlongPedestrianMode)

Spawned as type-12 vehicles with a Poisson mean of `AlongPedestrianPerHour/3600` per route per step. Their initial strip comes from the *walking pedestrian* blockage distribution, so they cluster near the kerb but stray into the carriageway realistically. Because they are vehicles, they follow, overtake and get overtaken like everything else, and they are weighted by `PedestrianWeight` when other road users decide how much to respect them.

Note the 20-step on/off toggle described in Section 6: footpath pedestrian traffic arrives in bursts, not steadily.

11.3 Roadside objects (ObjectMode)

Four types, each with an empirically fitted **mixture of Gaussians** for how far into the carriageway it protrudes (`constants.py`):

Type	Object	L × W (m)	Blockage (m)	Components	Target
1	standing pedestrian	0.7×0.7	0.21–6.00	5	60
2	parked car	4.5×1.7	0.70–6.00	6	76
3	parked rickshaw	3.0×1.0	0.56–6.00	5	60
4	parked CNG	2.6×1.3	0.95–4.67	4	18
5	<i>walking</i> pedestrian	—	0.50–7.75	2	(type-12 placement)

Table 7: Roadside object types and their empirical blockage distributions.

Target counts are $\lceil \text{rate} \times \text{TOTAL_NETWORK_ROAD_LENGTH} \rceil$ with `TOTAL_NETWORK_ROAD_LENGTH` 3.83 km.

- **Spawn probability** per link per step is $\text{factor} \times k \times (1 - \Phi(\text{currentCount}; \text{targetCount}, 10))$ with `factor` = 0.1 and $k = 3, 1, 4, 7$ for the four types. As the population approaches its target the Gaussian CDF drives the spawn rate towards zero, so counts self-regulate around the surveyed density instead of growing without bound.

- **Placement:** a random segment, a random longitudinal position in $[9, \text{length}-9]$, a random side, and a blockage depth drawn from the mixture. The object occupies every strip from the footpath up to its blockage depth — and is rejected outright if any of them lacks a 0.5 m clear gap.
- **Parking duration,** drawn uniformly: standing pedestrian 10–60 s; parked car 100–500 s; parked rickshaw 30–150 s; parked CNG 60–300 s. When it expires the object frees its strips and disappears.
- **Effect on traffic:** `is_object_in_proximity()` is the *first* condition in the movement decision, so an approaching object triggers a strip change before any car-following consideration — and a vehicle will never change strips *towards* an object.

12 Intersections & Signals

12.1 Turn paths are created lazily

There is no pre-computed turn table. The first time a vehicle wants to go from (link A, strip i) to (link B, strip j), `Processor._create_intersection_strip()` computes the straight-line path across the junction between the two strip centres in pixel space and registers it as an `IntersectionStrip` on the node. Subsequent vehicles reuse it. The exit strip is derived by `Vehicle.get_new_strip_index()`, which maps the strip proportionally between roads of different width — a vehicle in the third of twenty strips arrives in roughly the third of however many strips the next road has.

12.2 Signals

Turn paths entering the junction from the same link share an `IntersectionStripBundle`, and the bundle carries the signal. Exactly one bundle is green at a time; `constant_signal_change()` advances to the next bundle once $\text{simulationTime} - \text{timePassed} \geq \text{SignalChangeDuration}$.

With the shipped `SignalChangeDuration` 1, the green phase advances **every simulated second**. That is effectively a round-robin gate rather than a realistic signal plan — raise it (e.g. 30–60) for signal-timing studies.

`adaptive_signal_change()` — which extends green time based on the queue pressure on the approach — is implemented but **not called**; `_control_signal()` calls `constant_signal_change()`. Swapping the call is a one-line change.

12.3 Entering, traversing and leaving

- **Entering:** at the segment end, if a free strip exists in the entering segment and the bundle is green, the vehicle joins the node’s vehicle list, `isInIntersection` becomes true, and it releases its segment strips. If the signal is red, or the entry is blocked, or it would overlap a vehicle already in the junction, its speed is set to 0.
- **Traversing:** inside the junction there are no strips. Collision avoidance uses **separating-axis testing** on the four rotated corner points of each vehicle (`Node._is_colliding`). `_control_speed_in_intersection()` tries successively larger speeds and keeps the largest that does not overlap.
- **Deadlock escape:** a vehicle stuck for more than 40 steps sets `noForceMove = False`, which lets it add $1.5 \times \text{length}$ to its speed and jump through the blockage. Before that, `_try_changing_direction_in_intersection()` attempts every alternative exit strip.

- **Leaving:** `_move_vehicle_at_intersection_end()` finds a free strip in the next link, updates the route index, and resumes normal segment driving.

13 The Input File System — Scenario Specification

Every scenario is defined purely by plain-text files in `input/`. No code changes are required to model a new location.

13.1 `node.txt` — intersections and boundaries

```
1 21
2 0 0 0 16 18 17 14
3 1 317 26 4
4 2 436 20 19
5 3 675 14 20
6 12 0 0 15 14 13 12
```

Listing 3: Sample `node.txt`

Format: first line = node count; then `nodeId x y linkId1 linkId2 ...`

- **Boundary nodes** (exactly 1 link) are the entry/exit points of the network. `run_sim.py` generates demand between every ordered pair of these.
- **Interior nodes** (> 1 link) become signalised junctions. They conventionally carry $x = 0, y = 0$, because their true position is implied by the geometry of the links meeting there; only non-zero coordinates contribute to the view-centring calculation.

13.2 `link.txt` — road geometry

```
1 23
2 0 8 16 1
3 0 618 730 591 567 27
4 1 11 18 1
5 0 67 403 161 409 32
6 2 10 18 2
7 0 110 735 134 656 32
8 1 137 647 175 440 27
```

Listing 4: Sample `link.txt`

Link header: `linkId upNodeId downNodeId segmentCount`

Segment line: `segmentId startX startY endX endY width`

All values are metres. A link is a chain of straight segments, which is how curved roads are approximated. The **width drives strip capacity** via the formula in Section 2 — it is the single most important number for capacity.

13.3 `path.txt` — routes (generated)

```
1 110
2 1 2 4 5 9 19
3 1 3 4 5 9 20
```

Listing 5: Sample `path.txt`

Format: `srcNodeId dstNodeId linkId1 linkId2 ...`

13.4 demand.txt — traffic volume matrix

```
1 110
2 1 2 100
3 1 3 100
```

Listing 6: Sample demand.txt

Format: srcNodeId dstNodeId vehiclesPerHour

Remember the **+30 offset** applied at load time (Section 7). `demand.txt` and `path.txt` must be consistent: every demand row needs at least one matching route, or that demand is silently unusable.

13.5 parameter.txt — runtime configuration

One Name Value pair per line; unknown names are ignored, so you can leave comments. See Section 14.

13.6 Regenerating routes and demand

`run_sim.py` builds the node adjacency matrix from the link/node files, runs **Floyd–Warshall** to obtain all-pairs shortest paths, emits one route and one demand row per ordered boundary-node pair, then thins the demand according to `DemandType`:

DemandType	Accepted routes	Per-route rate
0 low	$\approx 1/3$ of boundary nodes	LowRate / acceptableNodes
1 medium	$\approx 2/3$	MediumRate / acceptableNodes
2 high	all	HighRate / acceptableNodes

The unthinned file is preserved as `input/demand_all.txt`.

```
1 python run_sim.py
```

14 Complete Parameter Reference

Defaults are the shipped values.

14.1 Run control

Name	Default	Meaning
SimulationEndTime	1800	simulated seconds to run
SimulationSpeed	1	GUI frame delay in ms
GUIMode	On	On opens the animation
TraceMode	Off	On replays a recorded <code>trace.txt</code> instead of simulating
DebugMode	Off	On writes per-entity traces to <code>debug/</code>
RandomSeed	0	ignored — see quirk 1

14.2 Geometry & display

Name	Default	Meaning
StripWidth	0.5	strip width in metres — the spatial resolution
FootpathStripWidth	0.5	width of the two footpath strips
PixelPerMeter	15	world-to-pixel scale (also used by junction geometry)
CenteredView	On	centre the view on the network centroid
DefaultTranslateX / Y	−1340 / −580	manual view offset when CenteredView Off

14.3 Behavioural models

Name	Default	Meaning
CF_model	0	car-following model 0–12 (Section 9)
DLC_model	0	lane-changing model 0–3 (Section 10)
BrakeHard	Off	add a hard gap-based speed clamp on top of the model
ConsiderMinimum	On	MY_MODEL: minimum over leaders instead of weighted average
DensityPercentage	50	MY_MODEL: density threshold for widening attention
PedestrianWeight	2	MY_MODEL: how strongly pedestrians restrain vehicles
MaximumSpeed	100.0	network speed limit — m/s headless, km/h in the GUI
ALPHA / BETA / ETA	0.92 / 0.1 / 0	KFTM calibration constants
TTC_Threshold	0.6	time-to-collision threshold for near-crash counting
PenaltyWait	On	collided vehicles wait out a penalty before removal
EncounterPerAccident	0.01	accident propensity (transformed at load; quirk 4)

14.4 Demand

Name	Default	Meaning
DemandType	2	0 low / 1 medium / 2 high — used by <code>run_sim.py</code>
LowRate / MediumRate / HighRate	200 / 600 / 1000	total veh/h for each level
VehicleGenerationRate	1	0 constant headway, 1 Poisson arrivals
SlowVehicle / MediumVehicle / FastVehicle	50 / 25 / 25	overwritten with 25/25/50 at load (quirk 2)

14.5 Side friction

Name	Default	Meaning
ObjectMode	On	generate parked vehicles and standing pedestrians
AcrossPedestrianMode	On	generate road-crossing pedestrians
AlongPedestrianMode	On	generate pedestrians walking along the road
AcrossPedestrianPerHour	400	crossing-pedestrian arrival rate per link
AlongPedestrianPerHour	50	along-road pedestrian arrival rate per route
AcrossPedestrianLimit	1	legacy — parsed but never read
AlongPedestrianPercentage	99	legacy — parsed but never read
PedestrianRandomLaneChangePercentage	20	chance a walker attempts a lateral move
PedestrianLeftBiasPercentage	40	of those, the chance it is towards the kerb

14.6 Signals & measurement

Name	Default	Meaning
SignalChangeDuration	1	seconds per green phase (Section 12)
NoOfRoutes	12	how many routes get their own per-route CSVs
ErrorMode	Off	enable sensor-error perception of gaps
FTMethod	1	1 = repeated readings, 2 = margin factor, 3 = both
NoOfReadings	2	readings per gap measurement when ErrorMode On
MFactor	0.0	safety margin multiplier on measurement bounds

15 Output Files & Metric Definitions

15.1 Standard output

Six lines are printed at the end of the run:

```
1 speed: 7.5465...
2 waiting time: 4.7762...
3 motorized speed: 9.2665...
4 motorized waiting time: 4.6145...
5 non-motorized speed: 2.1864...
6 non-motorized waiting time: 5.2801...
```

Metric	Definition	Unit
speed	mean over vehicles of (distance travelled ÷ time alive)	m/s
waiting time	mean over vehicles of the number of steps with zero speed	seconds
motorized ...	the same, restricted to types 3–11	
non-motorized ...	the same, restricted to types 0–2	

Type 12 (pedestrians) is excluded from all six. Note the unit mismatch with the CSVs: **stdout speeds are m/s, avg_speed_vehicle.csv is km/h.**

Accident: SimStep: ...Vehicle ...Pedestrian ... lines are also printed as they happen — counting these lines is currently the only way to get the accident total (quirk 5).

15.2 statistics/*.csv

All files are **appended**, one row per run, so a directory accumulates an experiment. Every per-type file has 13 comma-separated columns in vehicle-type order with a trailing comma. Empty cells print as NaN.

File	Row contents
avg_speed_vehicle.csv	mean speed per type, km/h
waiting_percentage_vehicle.csv	$100 \times \text{waitingTime} / \text{totalTravelTime}$ per type
generated_vehicles.csv	vehicles generated per type
agg_total_trip_complete.csv	completed trips per type
agg_avg_tt.csv	mean trip time per type, minutes
agg_avg_fuel.csv	mean fuel per completed trip per type, litres
agg_avg_collision.csv	collisions per type
agg_avg_accident.csv	accidents per type
agg_total_collision.csv	4 values: collisions, accidents, vehiclesGenerated, pedestriansGenerated
avg_tt<i>.csv	mean trip time per type for route i , minutes
fuel<i>.csv	mean fuel per type for route i
trip_complete<i>.csv	completed trips per type for route i
collisions<i>.csv	collisions per type for route i
flow.csv	one column per simulated minute: vehicles past the flow sensor
accident_log.csv	simStep, vehicleId, type, speed, acceleration, leaderType, leaderSpeed, leaderAcc

Per-route files exist for $i = 0 \dots \min(\text{NoOfRoutes}, \text{routeCount}) - 1$, indexed by the row order of demand.txt.

flow.csv is all zeros for the shipped network. Processor._update_flow() counts only vehicles crossing **950 m along link 0, segment 0, in the reverse direction** — all four values are hard-coded. Link 0's first segment in the Dhaka network is 165 m long, so the sensor is never reached. To use it, edit link_id, segment_id, sensor_distance and direction at the top of _update_flow() to a location that exists. The per-segment sensor used by the average-speed statistics is unrelated and sits at each segment's mid-point.

15.3 The fuel model

Vehicle.compute_fuel_consumption() implements an instantaneous power-demand model with a 1325 kg reference mass. Per time step, with v in km/h and a in m/s²:

$$R_t = Av + Bv^2 + Cv^3 + m a v$$

$$\Delta F = \begin{cases} \alpha + \beta v + \delta v^3 + \xi a v & R_t > 0 \\ f_{\text{idle}} & \text{otherwise} \end{cases}$$

with $A = 0.1326$, $B = 0.0027384$, $C = 0.0010843$, $\alpha = 0.365$, $\beta = 0.00114$, $\delta = 9.65 \times 10^{-7}$, $\xi = 0.0943$, $f_{\text{idle}} = 0.299$ g/s, converted to litres using a petrol density of 0.73722 g/cm³. Idling still burns fuel, which is why congestion shows up clearly in agg_avg_fuel.csv.

15.4 trace.txt

Written by the GUI, one block per rendered frame:

```
1 <simulationSpeed> <simulationEndTime> <acrossPedestrianMode>
2 Current Pedestrians
3 <x> <y> <inAccident>                                <- one per
   pedestrian
4 Current Vehicles
5 <x1> <x2> <x3> <x4> <y1> <y2> <y3> <y4> <red> <blue> <green>    <- one per
   vehicle
6 Current Objects
7 End Step
8 SimulationStep: <n>
9 ...
```

Listing 7: trace.txt structure

Coordinates are untransformed world pixels, so a trace is independent of the pan/zoom state at capture time. Set `TraceMode On` to replay one.

16 The Numeric Compatibility Layer

`dhakasim/javacompat.py` exists because Python's default arithmetic differs from the original implementation's in ways that **change simulation results**. This is not defensive programming; each item below was necessary to reproduce the reference output.

Helper	Why Python's default is wrong
jmin, jmax	Java's <code>Math.min/Math.max</code> propagate NaN ; Python's <code>min/max</code> silently return the non-NaN operand. Gipps braking produces NaN on every emergency stop, so this single difference removes emergency braking and inflates every speed statistic.
jround	<code>Math.round</code> rounds half up ; Python's <code>round</code> rounds half to even .
jint	<code>(int)</code> truncates towards zero and saturates at the 32-bit range; Python's <code>int()</code> truncates but does not saturate, and <code>//</code> floors.
jdiv, jsqrt, jlog, jexp, jpow	Java returns <code>Infinity/NaN</code> ; Python raises . The simulator divides by zero (empty statistics buckets, zero-length intervals) and takes sqrt of negatives (Gipps) routinely.
jmod	<code>%</code> on a negative dividend keeps the dividend's sign in Java, floors in Python.
jformat	<code>String.format("%.3f", x)</code> rounds the shortest round-tripping decimal half-up, not the exact binary value — so <code>%.2f</code> of 1.005 is 1.01, not 1.00. Non-finite values print as <code>NaN/Infinity</code> , which is why the CSVs contain literal <code>NaN</code> .
jstr	<code>Double.toString</code> — the exact form of the speed: ... lines that downstream scripts parse.
jto_radians	<code>Math.toRadians</code> computes $x/180 \cdot \pi$, not $x \cdot (\pi/180)$; the two differ in the last bit.
DOUBLE_MIN_VALUE	It is the smallest positive denormal, not the most negative double. The junction collision test seeds a maximum accumulator with it.
JavaRandom	A bit-exact 48-bit LCG <code>java.util.Random</code> , including <code>nextInt(bound)</code> 's rejection loop, <code>nextFloat</code> 's 24-bit draw, <code>nextGaussian</code> 's cached second value, and <code>nextDouble(origin, bound)</code> . A given seed therefore drives an identical simulation.
Color, lines_intersect	The slices of <code>java.awt</code> the drawing code needs.
PoissonDistribution, GammaDistribution	The two Apache Commons-Math samplers, same algorithms (multiplication method; Marsaglia–Tsang).

`tests/test_javacomp.py` pins all of this to values captured from a real JVM. **If that test fails, the simulation numerics have drifted** — run it after any change to the layer.

17 Known Behaviours & Quirks

These are long-standing behaviours inherited from the original implementation. They are preserved deliberately because they affect results, and each is commented at the point in the code where it happens. **Read this section before interpreting any output.**

1. **RandomSeed in parameter.txt is ignored.** `Utilities.initialize()` discards the file value and draws a fresh seed in $[0,101)$, so **runs are not reproducible**. The RNG underneath *is* seed-faithful, so for repeatable runs change the `RandomSeed` branch in `utilities.py:initialize` to `Parameters.seed = int(value)`. The GUI's seed field *does* apply what you type.
2. **Parameter parsing falls through.** A `DLC_model` line also sets the car-following model from the same value, and then assigns `SlowVehicle`; a `CF_model` line also assigns `SlowVehicle`. Because `parameter.txt` lists `DLC_model` before `CF_model`, the `CF_model` line wins — **keep that order**. Separately, `SlowVehicle/MediumVehicle/FastVehicle` are unconditionally overwritten with 25/25/50 at the end of parsing, so the file values have no effect on the vehicle mix.
3. **MaximumSpeed changes units between modes.** Headless reads it as m/s; the GUI option form reads it as km/h and divides by 3.6. The shipped 100.0 therefore caps speeds at 100 m/s headless but 27.78 m/s in the GUI — the same scenario yields different results in the two modes.
4. **Probability of Accident is transformed twice in the GUI.** Startup turns `EncounterPerAccident` 0.01 into 10000; the option form displays that 10000 and transforms it again, giving -9899. Nothing currently reads the result.
5. **Accidents never reach the counters.** `accident_log_all_vehicle_at_last()` calls `has_caused_accident()`, which *removes the pedestrian* as a side effect of detecting the strike. The later `accident_check_all_vehicles_at_last()` therefore finds nothing, `after_accident_to` never runs, and `agg_total_collision.csv` reports 0 collisions and 0 accidents — even while `Accident: SimStep: ...` lines are printed and `accident_log.csv` is populated. **Count the stdout lines, or the rows of accident_log.csv, for the real total.** As a consequence the collision penalty (`PenaltyWait`) and the gamma-distributed penalty time are also never applied.
6. **End-of-run double counting.** `_generate_statistics()` adds the travel and waiting time of every vehicle still on the road, then calls `calculate_statistics_at_end()`, which adds both again. Vehicles that had already completed their trip are counted once. So `waiting time` and `waiting_percentage_vehicle.csv` are biased upwards by the vehicles in flight at the end — a smaller effect the longer the run.
7. `Segment.get_accident_count()` **mutates**. The Java original is `return accidentCount++`, so reading the counter increments it. Preserved.
8. `statistics/` **is appended, never truncated**. Delete the folder between experiments or each file grows a row per run. This is intentional — it is how batch sweeps accumulate.
9. **Lost demand is not queued.** If no strip gap exists at the entry segment, the vehicle is discarded rather than held. At saturation the *offered* demand in `demand.txt` exceeds what enters the network; compare `generated_vehicles.csv` against the nominal rate to see how much was lost.
10. **assert statements were no-ops.** The original ran without `-ea`, so its assertions never fired; they are comments here rather than live checks.

18 How to Simulate a New Location

No code changes are required.

1. **Step 1 — Collect geometry.** From Google Maps, OpenStreetMap or GIS data for your target area (Mirpur-10, Farmgate, ...), read off intersection positions and road widths in metres. Pick any convenient local origin; coordinates are plain metres, not lat/lon.
2. **Step 2 — Write input/node.txt.** Number nodes from 0. Give each boundary node its real x y and the single link id attached to it. Give each interior junction 0 0 and the list of link ids meeting there.

```

1 5
2 0 0 0 0 1 2 3      <- interior junction, four approaches
3 1 500 0 0          <- boundary node at (500, 0)
4 2 0 500 1
5 3 -500 0 2
6 4 0 -500 3

```

3. **Step 3 — Write input/link.txt.** One header per link, then one line per straight segment. Split curves into several segments.

```

1 4
2 0 0 1 1
3 0 0 0 500 0 12.0    <- 12 m wide road from the junction to node 1
4 1 0 2 1
5 0 0 0 0 500 12.0
6 2 0 3 1
7 0 0 0 -500 0 12.0
8 3 0 4 1
9 0 0 0 0 -500 12.0

```

Check the width gives the strips you expect: $2 + \lfloor (12.0 - 1.0)/0.5 \rfloor = 24$ strips $\rightarrow$ 11 per direction.

4. **Step 4 — Generate routes and demand.**

```

1 python run_sim.py

```

This writes input/path.txt and input/demand.txt (and keeps the unthinned demand as input/demand_all.txt). Set DemandType, LowRate, MediumRate and HighRate in parameter.txt *before* running it — they control the output.

5. **Step 5 — Configure the run.** In input/parameter.txt set SimulationEndTime, the model choices, the side-friction switches, and either CenteredView On or a manual DefaultTranslateX/Y.

6. **Step 6 — Run.**

```

1 python run_dhakasim.py --gui

```

Watch it once in the GUI to confirm the geometry looks right and vehicles route sensibly, then switch to -headless for the measured runs.

Common pitfalls

- **Road too narrow for the widest vehicle:** a bus needs 5 strips per direction, so a direction with fewer than 5 usable strips will silently never accept buses.
- **Segment shorter than ≈ 18 m:** pedestrians and objects spawn only in $[9, \text{length} - 9]$, so shorter segments get no side friction at all.
- **A demand row with no matching route** in path.txt generates nothing.

19 Simulating Road Diversions & Strip Blocking

19.1 Full closure

Remove the link from `input/link.txt`, remove its id from the affected nodes in `input/node.txt`, and re-run `run_sim.py`. Floyd–Warshall re-routes the traffic around the closure automatically.

19.2 Road narrowing (construction, flyover works)

Reduce the `width` field of the affected segment(s) in `input/link.txt`. Strip count — and therefore capacity — follows directly. No route regeneration needed.

```
1 2 10 18 2
2 0 110 735 134 656 32      <- unchanged
3 1 137 647 175 440 18      <- narrowed from 27 m to 18 m
```

19.3 Static kerbside encroachment

Raise `AcrossPedestrianPerHour`, or leave `ObjectMode` On and let the empirical distributions place parked vehicles. To model a specific stretch as permanently obstructed, narrow that segment instead — it is more predictable than waiting for random objects.

19.4 Scripted obstacle (accident, dumped material)

Insert an `Object` at a chosen step. Add this to `Processor._run_at_each_time_step()`:

```
1 # blockage on link 3, segment 0, from step 500 onwards
2 if Parameters.simulation_step == 500:
3     link = self.link_list[3]
4     segment = link.get_first_segment()
5     self._generate_an_object(link, 2)      # 2 = parked car
```

Listing 8: Simple scripted blockage

For precise placement, construct it directly instead:

```
1 from .roadside_object import Object
2
3 if Parameters.simulation_step == 500:
4     link = self.link_list[3]
5     segment = link.get_first_segment()
6     obj = Object(
7         self.object_id, 2, Parameters.simulation_step, link, segment,
8         0,                                     # segment index
9         segment.get_length() / 2,             # longitudinal position, metres
10        False,                                # side: False = near footpath
11        2.5,                                  # blockage depth into the carriageway,
12        metres
13    )
14    self.object_id += 1
15    self.object_list.append(obj)
```

Listing 9: Precisely placed blockage

The constructor occupies the strips itself. Give it a long parking time by editing the duration draw in `roadside_object.py`, or it will clear after 100–500s.

19.5 Closing individual strips

There is no “closed strip” concept. Model it with a chain of stationary objects covering the length of the blocked stretch, or narrow the segment if the closure runs kerbside for its whole length.

20 Extension Recipes

20.1 Change the vehicle mix

Edit `Processor._distributed_vehicle_type()`. The bands are read against `slow_vehicle_percentage` / `medium_vehicle_percentage`, which are forced to 25/25/50 at load — so to change the coarse split you must edit the assignment at the end of `Utilities.initialize()`, not `parameter.txt` (quirk 2).

20.2 Add a vehicle type

1. Append width, length, speed and acceleration to the four tuples in `utilities.py`.
2. Increase `Constants.TYPES_OF_CARS` — every statistics array is sized from it, so the CSVs gain a column automatically.
3. Return the new type index from `_distributed_vehicle_type()`.
4. If it is non-motorised, note that the motorised/non-motorised split in `_generate_statistics()` is hard-coded as $i < 3$.

20.3 Add a car-following model

1. Add a member to `CAR_FOLLOWING_MODEL` in `parameters.py`.
2. Map its `CF_model` integer in `Utilities._apply_cf_model()`.
3. Write `_get_new_speed_<name>_model()` on `Vehicle`, following the existing shape: compute a free-flow speed v_a , a following speed v_b , and return `Utilities.precision2(jmax(0.0, jmin(v_a, v_b)))`.
4. Dispatch it in `Vehicle.get_new_speed()`.

Use `jmin/jmax`, not the built-ins, or NaN handling will differ from every other model.

20.4 Use adaptive signals

In `Processor._control_signal()`, swap `constant_signal_change` for `adaptive_signal_change` — it is already implemented and extends the green phase by the queue pressure on the approach, capped at 120s.

20.5 Per-vehicle trajectory output

`Processor.write_speed()` returns `False`. Return a predicate instead, e.g. `vehicle.get_vehicle_id() % 40 == 0` and `vehicle.get_type() != 12`, and each matching vehicle gets `statistics/speeds_<id>.csv` plus x/y trajectory files on removal. Note the trajectory is only populated in GUI mode, since the coordinates come from the drawing pass.

20.6 Make runs reproducible

In `utilities.py:initialize`, replace

```
1 elif name == "RandomSeed":
2     # the value in the file is deliberately ignored
3     Parameters.seed = JavaRandom().next_int_bound(101)
```

with

```

1 elif name == "RandomSeed":
2     Parameters.seed = int(value)

```

A negative value keeps the random behaviour. Note that the roadside-object and pedestrian-arrival samplers use their own generators, seeded from the clock — make those derive from `Parameters.seed` too if you need end-to-end determinism.

21 Performance

The shipped 1800-step Dhaka network (23 links, 110 routes, all side friction enabled) takes **7–8 minutes** on a desktop CPU. Cost is dominated by the per-vehicle, per-strip neighbour searches in `strip.py`, so runtime grows with the number of *concurrent* vehicles and is worse than linear in `SimulationEndTime`.

- Iterate at `SimulationEndTime` 120–300, then do measured runs at full length.
- `ObjectMode Off`, `AcrossPedestrianMode Off` and `AlongPedestrianMode Off` speed things up substantially — useful when debugging geometry rather than studying side friction.
- Batch sweeps parallelise trivially: give each run its own working directory (each needs its own `input/` and `statistics/`) and run them concurrently.
- The GUI keeps up at `SimulationSpeed 1`; drawing cost is proportional to the number of visible vehicles.

22 Provenance & Validation

This implementation was translated from the original Java simulator, one Python module per Java class, and validated against it. With a matched random seed both produce **byte-identical statistics/*.csv** and identical printed metrics, verified across:

Coverage	Result
All 13 car-following models (<code>CF_model</code> 0–12)	identical
All 4 lane-changing models (<code>DLC_model</code> 0–3)	identical
<code>BrakeHard</code> , <code>ConsiderMinimum</code> variants	identical
Crossing pedestrians, along-road pedestrians, roadside objects	identical, incl. the accident log
Route/demand generation (<code>run_sim.py</code>)	byte-identical output files
Vehicle / object / pedestrian draw geometry (<code>trace.txt</code>)	identical

The side-friction paths use unseeded generators in the original, so they were compared by seeding both sides identically in a scratch build. Unseeded, repeated runs of the two implementations agree within statistical noise (all metrics within 1.3 standard errors over 8 runs each).

22.1 Java class → Python module map

Java	Python
<code>Sim.java</code>	<code>run_sim.py</code>
<code>thesisfinal/DhakaSim.java</code>	<code>dhakasim/dhaka_sim.py</code> + <code>run_dhakasim.py</code>
<code>Processor.java</code>	<code>dhakasim/processor.py</code>
<code>Vehicle.java</code>	<code>dhakasim/vehicle.py</code>

Java	Python
Strip.java	dhakasim/strip.py
Segment.java	dhakasim/segment.py
Node.java	dhakasim/node.py
Object.java	dhakasim/roadside_object.py (class Object)
Pedestrian.java	dhakasim/pedestrian.py
Utilities.java	dhakasim/utilities.py
Parameters.java	dhakasim/parameters.py
Constants.java	dhakasim/constants.py
Statistics.java	dhakasim/statistics.py
VehicleStats.java	dhakasim/vehicle_stats.py
Link/Path/Demand.java	dhakasim/link.py, path.py, demand.py
IntersectionStrip(Bundle).java	dhakasim/intersection_strip(_bundle).py
LinkSegmentOrientation.java	dhakasim/link_segment_orientation.py
NormalDistribution.java	dhakasim/normal_distribution.py
Point2D.java, SIGNAL.java	dhakasim/point2d.py, signal.py
DhakaSimFrame/Panel.java, OptionPanel.java	dhakasim/gui.py
—	dhakasim/javacompat.py (Java numeric semantics)

Method names are the Java names in `snake_case` (`getDistanceInSegment` → `get_distance_in_segment`); private fields keep a leading underscore; Java `static` fields are class attributes.

23 Quick Reference

23.1 Commands

```

1 python run_dhakasim.py           # honour GUIMode in parameter.txt
2 python run_dhakasim.py --headless # force batch mode
3 python run_dhakasim.py --gui      # force the animation
4 python run_sim.py                 # regenerate path.txt + demand.txt
5 python tests/test_javacompat.py   # verify numeric fidelity

```

23.2 Key constants

Further Reading

1. M. M. Mushfiq et al., “*To Lane or Not to Lane?*—Comparing On-Road Experiences in Developing and Developed Countries Using a New Simulator RoadBird,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 8, pp. 8486–8498, Aug. 2024. [doi:10.1109/TITS.2024.3406731](https://doi.org/10.1109/TITS.2024.3406731)
2. A. M. S. Rumi et al., “A Tale of Side Friction Elements in Dhaka City: From Modeling Real Cases to Revealing Impacts Through Event-Based Simulation,” *IEEE Access*, vol. 13, pp. 37344–37360, 2025. [doi:10.1109/ACCESS.2025.3544511](https://doi.org/10.1109/ACCESS.2025.3544511)

Scenario Goal	What to change	Code edit?
New map / location	input/node.txt, input/link.txt, then run_sim.py	No
Road diversion / closure	remove link from link.txt + node.txt, re-run run_sim.py	No
Lane reduction / narrowing	reduce width in link.txt	No
Adjust traffic volume	input/demand.txt, or DemandType + *Rate then run_sim.py	No
Change car-following model	CF_model in parameter.txt	No
Change lane-changing model	DLC_model in parameter.txt	No
Change signal timing	SignalChangeDuration in parameter.txt	No
Turn side friction on/off	ObjectMode, AcrossPedestrianMode, AlongPedestrianMode	No
Change pedestrian volumes	AcrossPedestrianPerHour, AlongPedestrianPerHour	No
Change strip resolution	StripWidth in parameter.txt	No
Scripted obstacle / accident	Processor._run_at_each_time_step()	Yes
Change vehicle type mix	Processor._distributed_vehicle_type()	Yes
Add a vehicle type	utilities.py tuples + Constants.TYPES_OF_CARS	Yes
Add a car-following model	parameters.py, utilities.py, vehicle.py	Yes
Adaptive signals	Processor._control_signal()	Yes
Reproducible runs	Utilities.initialize() RandomSeed branch	Yes

Table 9: Quick reference table for scenario customisations.

Constant	Value	Location
TIME_STEP	1.0 s	constants.py
THRESHOLD_DISTANCE	0.5 m standstill gap	constants.py
MARGIN	1.0 m from segment end	vehicle.py
TYPES_OF_CARS	13	constants.py
PEDESTRIANS_ALONG_THE_ROAD_TYPE	12	constants.py
max braking	-6 m/s^2 (all types)	vehicle.py
SAFE_TIME_GAP	0.0 s	vehicle.py
LAMBDA	0.78 (gap acceptance)	vehicle.py
TIME_WINDOW / GAP_WINDOW	4 / 3 steps of memory	vehicle.py
SIDE_STRIPS_TO_CONSIDER	1	parameters.py
TOTAL_NETWORK_ROAD_LENGTH	3.83 km	constants.py

Table 10: Constants most often needed when reading the code.